

AWS DynamoDB

04-data-engineering/dynamodb

Overview

DynamoDB is AWS's managed key value and document database. It is designed for high scale, predictable low latency access patterns, and minimal operational overhead.

In ML systems, DynamoDB is often useful at the serving edge where request latency matters and the access pattern is known ahead of time.

Core Concepts

- The primary design unit is the table with a partition key and optional sort key. Secondary indexes support alternate query patterns, but they should be designed intentionally.
- Capacity can be provisioned or on demand, and features such as TTL, streams, and transactions extend the base storage model for event driven and operational use cases.
- Data modeling starts from access patterns, not from trying to normalize relationships the way you would in a relational schema.

How It Works

- An application writes or reads items by key, query, or scan, though scans should be treated cautiously because they are rarely the right production access pattern.
- Streams can publish item changes to downstream systems, which is useful for event driven enrichment or cache invalidation flows.
- Global tables or replication patterns can support multi region designs when low latency or resilience requirements justify the complexity.

AI and ML Relevance

- DynamoDB is a strong fit for feature lookups, prediction caches, idempotency records, session state, or metadata that must be served quickly during inference.
- It is also useful when upstream ML results need to be materialized into an application friendly structure for user facing workloads.

Operational Notes

- Hot partitions are a common failure mode. If the partition key concentrates too much traffic, the table can suffer throttling even when the aggregate capacity seems sufficient.
- Model the access pattern first and test it under realistic request skew. A clean schema on paper is not enough if runtime traffic clusters badly.
- Use scans sparingly, watch item sizes, and understand consistency choices so performance behavior is deliberate rather than accidental.

What To Remember

- DynamoDB rewards access pattern driven design and punishes generic relational instincts.
- Low latency comes from the right key design, not from magical auto scaling alone.
- For inference systems, it is often the supporting state store, not the place where large training datasets belong.